

LUMIÈRE SKIN CLINIC — ADMIN PANEL TECHNICAL DOCUMENT

Project: Skin Clinic Appointment Booking System **Module:** Admin / Staff Dashboard ("Admin Console") **Purpose:** Complete technical reference for the admin panel. This document is written so that an AI or a new developer can rebuild the admin panel from scratch with the same look, feel, behavior, and data model as the current production system.

TABLE OF CONTENTS

- 1. Executive Overview
 - 2. Technology Stack
 - 3. High-Level Architecture
 - 4. Folder / Directory Structure
 - 5. Authentication and Session Handling
 - 6. Routing, Role Guards, and Layout Shell
 - 7. Navigation Configuration (Sidebar + Command Palette)
 - 8. Design System (Colors, Typography, Spacing, Shadows)
 - 9. Reusable Dashboard Components
 - 10. Page-by-Page Specification
 - Dashboard Home
 - Today's Bookings
 - Appointments
 - OPD Schedule
 - Payments
 - Patients
 - Enquiries
 - Reviews
 - Services
 - Gallery
 - Staff Manage
 - Admin Manage
 - Settings
 - 11. Backend API Reference
 - 12. Database Schema
 - 13. Business Workflows (Booking, Refund, Birthday Cron, etc.)
 - 14. Third-Party Integrations
 - 15. Environment Variables
 - 16. Suggested Implementation Order
-

1. EXECUTIVE OVERVIEW

The admin panel is a single-page React 18 application that runs alongside the public-facing clinic website. It serves two user roles — `admin` and `staff` — and gives the clinic operational control over appointments, patients, services, payments, gallery, reviews, enquiries, OPD (clinic-hour) schedule, and staff/admin user accounts.

Two principles drive the design:

1. **Enterprise SaaS aesthetic.** Slate neutrals carry the layout; a single blue (#2563eb) is the only brand accent. Borders, not shadows, define cards. The console feels like a modern professional tool (think Linear / Stripe Dashboard), not a colorful consumer product.
2. **Operational safety.** Completed appointments are locked. The last admin cannot be deleted. Refunds and cancellations have explicit confirmation flows. The system prefers small reversible actions over bulk destructive ones.

The console is **mounted at** `/dashboard` and the public site lives at `/`. Users sign in at `/login`. Authentication is JWT-in-httpOnly-cookie, with a Redux store (`authSlice`) holding the current-user object on the client.

2. TECHNOLOGY STACK

Frontend

Layer	Tool	Version-style
Framework	React	18.x
Build tool	Vite	latest
Routing	react-router-dom	v6
State	Redux Toolkit	latest
HTTP	axios	with interceptors
Styling	Tailwind CSS	+ custom @layer components
Icons	react-icons/fi (Feather), Font Awesome (legacy pages)	—
Charts	Recharts	for analytics
Animation	framer-motion	for drawers, command palette, dropdowns
Toasts	react-hot-toast	success/error feedback
Date/Locale	en-IN / IST timezone everywhere	—

Backend

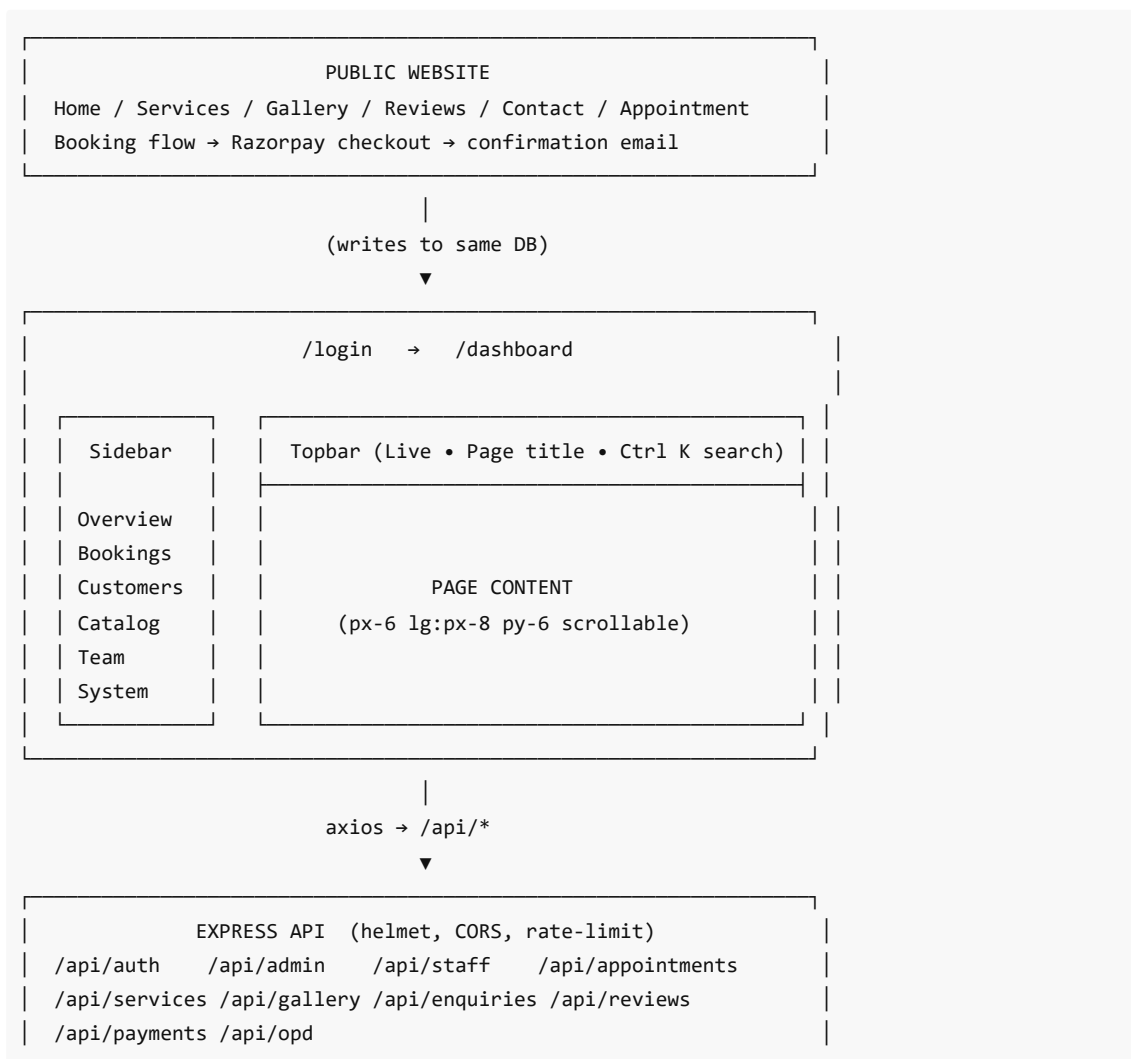
Layer	Tool
Runtime	Node.js
HTTP framework	Express
Database	MySQL 8.0+ (InnoDB, utf8mb4)
Auth	JWT in httpOnly cookie (token) + optional Authorization: Bearer fallback
File upload	multer (in-memory) + sharp (image compression)

Password hashing	bcrypt, 10 rounds
Email	nodemailer
Payments	Razorpay (orders + refunds + HMAC signature verification)
Scheduling	node-cron (birthday emails @ 09:00 IST)
Security	helmet, express-rate-limit, CORS allowlist

File Storage

- **Service images:** Cloudinary (legacy — may be migrating to local disk).
- **Gallery images:** Local disk under `backend/public/uploads/gallery/<sha1>.webp`, content-hashed; sharp-compressed when `>1 MB` or `>1920px` on the longest side.
- **URL imports** (paste a remote image URL): optional "mirror" mode downloads, compresses, and stores locally; otherwise the original URL is stored as-is.

3. HIGH-LEVEL ARCHITECTURE



▼
MySQL (users, patients, appointments, services,
doctors, payments, enquiries, reviews, gallery,
opd_defaults, opd_schedules, birthday_email_logs)

▼
Razorpay • Nodemailer SMTP • Cloudinary

The admin panel is purely a client of the same REST API the public site uses. **Role-based access is enforced both client-side** (route guards + visibility of nav items) **and server-side** (`isAuth` + `authorizeRoles` middleware on each endpoint).

4. FOLDER / DIRECTORY STRUCTURE

```
frontend/  
├─ src/  
│   ├── App.jsx                # Root routes; loads current user on mount  
│   ├── main.jsx               # Vite entry; wires Redux + Toaster  
│   ├── api/axios.js           # Single axios instance, baseURL + interceptors  
│   ├── store/authSlice.js     # User, loginThunk, logoutThunk, loadCurrentUser  
│   ├── routes/  
│   │   ├── ProtectedRoute.jsx # Auth gate (waits for /me)  
│   │   └─ RoleBasedRoute.jsx  # Role gate (admin-only pages)  
│   ├── styles/index.css       # Tailwind + custom dash-* utility classes  
│   ├── utils/formatters.js    # formatINR, formatTime12, formatDateLong, todayISO  
│   ├── components/  
│   │   ├── layout/PublicLayout.jsx # Public site shell  
│   │   ├── common/              # StatusBadge, Loader, etc.  
│   │   └─ dashboard/  
│   │       ├── DashboardLayout.jsx # Shell (sidebar + topbar + outlet)  
│   │       ├── Sidebar.jsx  
│   │       ├── Topbar.jsx  
│   │       ├── navConfig.jsx      # Single source of truth for nav items  
│   │       ├── PageHeader.jsx  
│   │       ├── Breadcrumbs.jsx  
│   │       ├── StatCard.jsx       # Primary KPI tile  
│   │       ├── MetricTile.jsx     # Secondary KPI tile (smaller)  
│   │       ├── CommandPalette.jsx # Ctrl+K spotlight  
│   │       ├── NotificationPanel.jsx # Topbar bell dropdown  
│   │       ├── EmptyState.jsx  
│   │       ├── Skeleton.jsx       # Shimmer placeholders  
│   │       ├── ExportButtons.jsx  # PDF / Excel / CSV wizard  
│   │       ├── NewAppointmentModal.jsx  
│   │       ├── CancelAppointmentModal.jsx  
│   │       ├── EnquiryDetailsDrawer.jsx  
│   │       ├── PatientDetailsDrawer.jsx  
│   │       └─ charts/  
│   │           └─ ChartCard.jsx
```

```

| | | | | AnalyticsCharts.jsx
| | | | |
| | | | | pages/
| | | | | | auth/Login.jsx
| | | | | | public/...
| | | | | | dashboard/
| | | | | | | DashboardHome.jsx
| | | | | | | TodayBookings.jsx
| | | | | | | Appointments.jsx
| | | | | | | OpdSchedule.jsx
| | | | | | | Patients.jsx
| | | | | | | Enquiries.jsx
| | | | | | | ReviewsManage.jsx
| | | | | | | Payments.jsx
| | | | | | | ServicesManage.jsx
| | | | | | | GalleryManage.jsx
| | | | | | | StaffManage.jsx
| | | | | | | AdminManage.jsx
| | | | | | | Settings.jsx
| | | | |
| | | | | tailwind.config.js # Brand palette + dash-* alias colors
| | | | | postcss.config.js
| | | | |
| | | | | backend/
| | | | | | index.js # Express bootstrap (middleware, routes, cron)
| | | | | | config/ # razorpay.js, mailer.js, db.js
| | | | | | routes/ # auth, admin, staff, appointments, services,
| | | | | | | # enquiries, reviews, gallery, payments, opd
| | | | | | controllers/ # one per route file
| | | | | | middleware/ # auth, role, upload (multer), error
| | | | | | model/ # one file per table; thin query helpers
| | | | | | jobs/birthday.job.js # Daily 09:00 IST cron
| | | | | | utils/imageStorage.js # sharp + content-hash file writer
| | | | | | public/uploads/ # Served at /uploads/*

```

5. AUTHENTICATION AND SESSION HANDLING

Token strategy

- Server signs a **JWT** containing `{ id }` with `process.env.JWT_SECRET`.
- Token is sent **both** as an `httpOnly` cookie (`token` , 7-day expiry, `SameSite=Lax` , `Secure` in production) **and** in the response body. The client stores the body token in `localStorage` as a fallback for browsers that block third-party cookies.
- Axios attaches `Authorization: Bearer <token>` from `localStorage` on every request. The cookie travels automatically because the axios instance is created with `withCredentials: true`.

Login flow

1. POST `/api/auth/login` with `{ email, password }`.
2. Server validates, bcrypt-compares against `users.password_hash`, rejects inactive users.
3. On success: sets cookie, returns `{ user, token }`. Frontend stores user in Redux and token in `localStorage`.

4. After login, the user is redirected to `/dashboard` (or to the `state.from` route they tried to access before login).

Logout flow

`POST /api/auth/logout` clears the cookie. The client also removes the `localStorage` token and resets `state.auth.user = null`.

loadCurrentUser (boot-time refresh)

On every page load, `App.jsx` dispatches `loadCurrentUser` which calls `GET /api/auth/me`. This populates `state.auth.user` from the cookie if one exists. Until the request resolves, `initialized = false`. `ProtectedRoute` renders a `Loader` during this phase to avoid bouncing logged-in users to `/login` on first paint.

Redux `authSlice` shape

```
state.auth = {
  user: { id, full_name, email, role, mobile, gender, dob, is_active, profile_image } |
  null,
  initialized: boolean, // true once /me has resolved or rejected
  loading: boolean, // true during login/logout/me
  error: string | null // last login error message
}
```

Selectors: `selectUser`, `selectIsAuthenticated`, `selectRole`, `selectAuthInitialized`, `selectAuthLoading`, `selectAuthError`.

6. ROUTING, ROLE GUARDS, AND LAYOUT SHELL

Route tree (`src/App.jsx`)

```
/
  PublicLayout
  /about /services /services/:slug /gallery /reviews /contact
  /appointment /appointment/confirmation/:id

/login
  Login

/dashboard
  ProtectedRoute → DashboardLayout
  index          DashboardHome          (both roles)
  /today         TodayBookings           (both roles)
  /appointments  Appointments            (both roles)
  /opd           OpdSchedule             (both roles)
  /gallery       GalleryManage           (both roles)
  /enquiries     Enquiries               (both roles)
  /reviews       ReviewsManage           (both roles)
  /settings      Settings                (both roles)

(RoleBasedRoute allow=['admin'])
```

/patients	Patients	(admin only)
/payments	Payments	(admin only)
/services	ServicesManage	(admin only)
/staff	StaffManage	(admin only)
/admins	AdminManage	(admin only)
*	Navigate("/")	

ProtectedRoute

- If `!initialized` → render a centered Loader on `dash-body` .
- If `!user` → `<Navigate to="/login" state={{ from: location }} replace />` .
- Otherwise → `<Outlet />` .

RoleBasedRoute

- Receives `allow` prop, e.g., `['admin']` .
- If `!allow.includes(user.role)` → `<Navigate to="/dashboard" replace />` .
- Otherwise → `<Outlet />` .

DashboardLayout

The shell that all dashboard pages render inside.

```
<div className="min-h-screen bg-[#f7f8fa] text-slate-700 font-sans">
  <Sidebar collapsed={collapsed} onToggleCollapsed={toggleCollapsed} />
  <div className={collapsed ? 'p1-[4.5rem]' : 'p1-64'}>
    <Topbar onOpenPalette={() => setPaletteOpen(true)} />
    <main className="px-6 lg:px-8 py-6">
      <Outlet />
    </main>
  </div>
  <CommandPalette open={paletteOpen} onClose={...} />
</div>
```

Key behaviors:

- The sidebar is **fixed left**, full height.
- The main content column is offset by `p1-64` (256px) when expanded, `p1-[4.5rem]` (72px) when collapsed.
- Sidebar collapsed state is persisted in `localStorage` under `dash:sidebar-collapsed` .
- A global `keydown` listener toggles the Ctrl+K / Cmd+K command palette.
- App background: `#f7f8fa` (very pale slate). Body text: `slate-700` (`#334155`).

7. NAVIGATION CONFIGURATION

`src/components/dashboard/navConfig.jsx` is the **single source of truth** for nav items. Both the Sidebar and the Ctrl+K Command Palette consume the same `NAV_SECTIONS` array. A `roles` field per item (optional) limits visibility.

```

NAV_SECTIONS = [
  { label: 'Overview', items: [
    { to: '/dashboard',          label: 'Dashboard',          icon: FiGrid,      end: true
  },
  ]},
  { label: 'Bookings', items: [
    { to: '/dashboard/today',     label: "Today's Bookings", icon: FiZap, live: true },
    { to: '/dashboard/appointments', label: 'Appointments',   icon: FiCalendar },
    { to: '/dashboard/opd',       label: 'OPD Schedule',   icon: FiClock },
    { to: '/dashboard/payments',  label: 'Payments',       icon: FiCreditCard, roles:
  ['admin'] },
  ]},
  { label: 'Customers', items: [
    { to: '/dashboard/patients',  label: 'Patients',       icon: FiUsers, roles:
  ['admin'] },
    { to: '/dashboard/enquiries', label: 'Enquiries',      icon: FiMessageSquare },
    { to: '/dashboard/reviews',  label: 'Reviews',       icon: FiStar },
  ]},
  { label: 'Catalog', items: [
    { to: '/dashboard/services',  label: 'Services',       icon: FiPackage, roles:
  ['admin'] },
    { to: '/dashboard/gallery',   label: 'Gallery',        icon: FiImage },
  ]},
  { label: 'Team', items: [
    { to: '/dashboard/staff',     label: 'Staff',          icon: FiUser,   roles:
  ['admin'] },
    { to: '/dashboard/admins',    label: 'Admins',         icon: FiShield, roles:
  ['admin'] },
  ]},
  { label: 'System', items: [
    { to: '/dashboard/settings',  label: 'Settings',       icon: FiSettings },
  ]},
];

canSee(item, role) => !item.roles || item.roles.includes(role);

```

Sidebar specification

- **Width:** `w-64` expanded / `w-[4.5rem]` (72 px) collapsed; smooth `transition-[width] duration-200`.
- **Surface:** `bg-white border-r border-slate-200`.
- **Brand block** at top (height 57 px, bottom-border):
 - `36×36` `bg-slate-900` rounded-lg tile with a white truck icon (`MdLocalShipping`) — a placeholder mark; replace with your logo.
 - Wordmark **"LUMIÈRE"** in `text-[15px] font-black tracking-wider text-slate-900`.
 - Subtitle **"Admin Console"** in `text-[10px] font-medium uppercase tracking-[0.14em] text-slate-400`.
- **Collapse button:** a `24×24` circular tab that sits half-outside the right border of the sidebar at `top:42px`, white with slate border, chevron rotates with state.

- **Section labels:** `text-[10px] font-semibold uppercase tracking-[0.14em] text-slate-400`, padding `px-4 pt-3 pb-1`. Hidden when collapsed; replaced by a thin `border-t border-slate-100` separator.
- **Nav item (idle):** `text-slate-600`, hover `bg-slate-50 hover:text-slate-900`, icon `text-slate-400`, font `text-[13px] font-medium`, padding `mx-2 px-3 py-2 rounded-md`.
- **Nav item (active):** `bg-blue-50 text-blue-700`, icon `text-blue-600`, plus a 2px-wide accent bar on the left of the item (`absolute left-0 top-1.5 bottom-1.5 w-[2px] rounded-r bg-blue-600`).
- **Badges next to nav items** (live counts from `/admin/dashboard-stats` or `/staff/dashboard-stats`, refreshed every 60 s):
 - `live: true` items use `bg-red-500 text-white animate-pulse`.
 - Active item: `bg-blue-100 text-blue-700`.
 - Idle: `bg-slate-100 text-slate-500`.
 - Display `99+` if count > 99.
- **Footer:** two links separated by `border-t border-slate-200 p-2`:
 1. **Back to Site** → `/`, slate.
 2. **Logout** → dispatches `logoutThunk()`, `text-red-600 hover:bg-red-50`.

Topbar specification

- **Surface:** `bg-white border-b border-slate-200 px-8 py-3`, sticky to the top with `z-30`.
- **Left cluster:**
 - **Live dot** — 8×8 emerald-500 pulsing ring (`.dash-live-dot`, 2s ease-in-out).
 - **"Live"** in `text-[11px] font-semibold tracking-wider uppercase text-emerald-600`.
 - Vertical divider (`text-slate-300`).
 - **Page title** derived from `PATH_TITLES[location.pathname]`, `text-[14px] font-semibold text-slate-900`.
- **Center search bar (opens command palette):** `flex-1 max-w-[460px]`, `bg-slate-50` idle / `bg-white` on hover with a `border-blue-300` hover ring. Contains a search icon, placeholder "Search or jump to a page...", and a `Ctrl K` `<kbd>` chip.
- **Right cluster:**
 - 36×36 rounded-lg **bell button** with red `unread` dot if any actionable count > 0 (sum of `pending_count + new_enquiries + pending_reviews + pending_payment_count`).
 - **Notification panel** (NotificationPanel) opens below the bell.
 - **User chip:** name/email stack + 36×36 initials avatar (`bg-slate-100 text-slate-700`) + role pill (`ADMIN` in `bg-blue-50 text-blue-700`, `STAFF` in `bg-slate-100 text-slate-600`).
 - User chip opens a dropdown (`framer-motion`) with: Profile & Settings, Back to Site, Logout.

8. DESIGN SYSTEM

8.1 Color Palette

All admin-panel UI uses Tailwind's native **slate** and **blue** scales, with semantic aliases defined in `tailwind.config.js`:

Token	Hex	Usage
<code>dash-bg</code>	<code>#f7f8fa</code>	App background

dash-line	#e5e7eb (slate-200)	Default border
dash-text	#334155 (slate-700)	Body text
dash-mute	#64748b (slate-500)	Labels, muted text
admin	#2563eb (blue-600)	Single brand accent
admin-deep	#1d4ed8 (blue-700)	Active / pressed
admin-soft	#eff6ff (blue-50)	Active-nav tint, soft highlights
admin-ink	#1e40af (blue-800)	Emphasis text

Status color mapping (used by `StatusBadge` , badges in tables, donut chart):

Status	Background	Text
paid / completed / approved	#ecfdf5	#047857 (emerald-700)
pending / contacted / rescheduled	#fffbeb	#b45309 (amber-700)
failed / cancelled / rejected	#fef2f2	#b91c1c (red-700)
refunded / no_show / closed	#f1f5f9	#475569 (slate-600)
confirmed / new / online	#eff6ff	#1d4ed8 (blue-700)
offline / cash	#eef2ff	#4338ca (indigo-700)

Accent tones for StatCard / MetricTile icon badges: blue, emerald, amber, indigo, teal, slate, rose. Each maps to a `bg-{tone}-50 text-{tone}-600` pair.

Public-site palette (for reference — used on public pages, NOT in the dashboard):

cream:	#faf7f2	blush:	#f5ede6
rose:	#d4a09a	gold:	#b8935a
gold-light:	#e8d5b7	sage:	#8fa88a
charcoal:	#2c2420	brown:	#5c3d2e
muted:	#8a7a70	ink:	#1a1008

8.2 Typography

Family	Tailwind alias	Use
Inter	<code>font-sans / font-body</code>	Body / dashboard text
Playfair Display	<code>font-heading</code>	Public-site headings
system-ui / Segoe UI fallback	(built-in)	Dashboard via <code>font-sans</code>

The dashboard uses **system-sans by default** (`ui-sans-serif`, `system-ui`, `-apple-system`, "Segoe UI", Roboto, ...) for a native SaaS feel, not Inter.

Typographic scale (dashboard):

Element	Size	Weight	Color
Page title (H1)	text-2xl (24 px) / font-semibold / tracking-tight	600	slate-900
Page subtitle	text-[13px]	400	slate-500
Card title	text-[15px] font-semibold	600	slate-900
Section label (uppercase)	text-[10px] font-semibold tracking-[0.14em] uppercase	600	slate-400
Stat number	text-[20px] font-semibold tabular-nums	600	slate-900
Body / table cell	text-[13px]	400	slate-700
Muted meta	text-[11px]	400	slate-500
Table TH	text-[11px] font-semibold uppercase tracking-wider	600	slate-500 (on bg-slate-50)
<kbd> chip	text-[10px] font-semibold font-mono	—	slate-500, bg-white border border-slate-200

8.3 Spacing & Layout Grid

- Outer page padding: px-6 lg:px-8 py-6 .
- Vertical rhythm inside pages: space-y-6 between major sections.
- Cards: rounded-xl border border-slate-200 bg-white . Inner padding p-5 (header), p-4 (filter/metric tiles).
- Stat row: grid grid-cols-1 sm:grid-cols-2 xl:grid-cols-4 gap-4 .
- Filter bar grid: grid grid-cols-1 md:grid-cols-12 gap-3 . Typical column spans: search=5, two selects=2 each, range=2, reset=1.
- Table cells: px-5 py-3.5 , rows divide-y divide-slate-100 , hover bg-slate-50 .

8.4 Borders, Shadows, Radii

- **Avoid shadows** for cards; use a single 1px border-slate-200 . Reserve shadows for floating surfaces (dropdowns, drawers, modals).
- Radii: 6 px (rounded-md) for small chips, 8 px (rounded-lg) for buttons / icon tiles, 12 px (rounded-xl) for cards.
- For drawers and modals, a backdrop is rendered as bg-slate-900/40 backdrop-blur-sm (or the legacy bg-[#1a1f36]/40).

8.5 Custom CSS utilities

Defined in `src/styles/index.css` under `@layer components` and `@layer utilities` :

Class	Purpose
<code>.dash-card</code>	White, slate-200 border, 0.75rem radius. No shadow.
<code>.dash-card-header</code>	Flex header with bottom-border border-slate-100.

<code>.dash-table</code>	TH uppercase 11px slate-500 on bg-slate-50; TD slate-700 13px; row hover slate-50.
<code>.dash-input</code>	8 px radius, slate-200 border, blue-300 focus border, blue-50 focus ring.
<code>.dash-label</code>	11px uppercase tracking-wide slate-500.
<code>.dbtn / .dbtn-primary / .dbtn-secondary / .dbtn-danger / .dbtn-ghost</code>	Button variants. Primary = blue-600 → blue-700. Secondary = white with border. Danger = white with red text + red-200 border.
<code>.queue-badge</code>	36×36 rounded-lg blue-50 / blue-600 with mono tabular-nums.
<code>.dash-live-dot</code>	8×8 emerald-500 with pulsing ring (<code>livePulse 2s</code>).
<code>.dash-empty</code>	Centered 64px empty state with icon and 13px slate text.
<code>.dash-section-label</code>	10px uppercase letter-spaced sidebar section header.
<code>.dash-kbar-overlay / .dash-kbar</code>	Command palette backdrop and modal panel.
<code>.skel / .skel-text / .skel-tile / .skel-row</code>	Shimmer skeletons (<code>skelShimmer 1.4s</code>).
<code>.dash-scroll</code>	Thin scrollbar that fades in on hover (sidebar nav, internal scrollers).
<code>.dash-drawer-overlay / .dash-drawer</code>	Right slide-in drawer.
<code>.badge / .badge-{value}</code>	Status pills mapped by value. See §8.1 for color mapping.

9. REUSABLE DASHBOARD COMPONENTS

9.1 PageHeader

Standard page header. Two rows:

1. (Optional) `<Breadcrumbs />` auto-derived from URL.
2. Flex row: title + subtitle on the left, `{children}` action buttons on the right (flex-wrap).

Title: `text-2xl font-semibold text-slate-900 tracking-tight` . Subtitle: `text-[13px] text-slate-500 mt-1` .

Props: `title` , `subtitle` , `crumbs` (boolean or array), `children` .

9.2 Breadcrumbs

Auto-derives crumbs from `useLocation().pathname` . Uses a `SEGMENT_LABEL` map to humanize URL segments (`appointments` → `Appointments` , `today` → `Today's Bookings` , etc.). Renders `FiHome` + chevron + crumb chain; last crumb is bold and non-link.

9.3 StatCard (primary KPI tile)

LABEL	[icon]
123	↑ +12%

Props: `accent` (blue/emerald/amber/indigo/teal/slate/rose), `icon` or `iconNode`, `label`, `value`, `trend`, `trendTone`.

Container: `bg-white rounded-xl border border-slate-200 p-5`. Icon badge: `w-8 h-8 rounded-lg bg-{tone}-50 text-{tone}-600`. Value: `text-[20px] font-semibold tabular-nums text-slate-900`. Trend pill: `bg-emerald-50 text-emerald-700` (or amber/rose).

9.4 MetricTile (secondary KPI tile)

Compact horizontal flex variant of StatCard for the secondary metric row beneath primary stat cards. Same accent palette.

9.5 Skeleton

`SkeletonText({width, className})`, `SkeletonTile()`, `SkeletonRow()`, and a composed `DashboardHomeSkeleton()` that renders a placeholder header + stat grid + metric row + table — the exact layout of `DashboardHome` so first paint doesn't shift.

9.6 EmptyState

Centered illustration card: `12×12 bg-slate-100` rounded square holding a `24px slate-400` icon, then a `15 px slate-900` title, then a `13 px slate-500` description, then an optional action slot. Used at the bottom of zero-result tables.

9.7 CommandPalette (Ctrl+K)

Spotlight modal that lists `NAV_SECTIONS` (filtered by role and query). Keyboard: `↑/↓` navigate items, `Enter` selects, `Esc` closes. Animated with `framer-motion` (opacity + 14ms scale). The input auto-focuses on open. Active item: `bg-blue-50 text-blue-700`. Sections use `text-[10px] font-semibold uppercase tracking-[0.14em] text-slate-400`.

9.8 NotificationPanel

Topbar bell dropdown (360 px wide, max 70vh). On open, fetches `/role/dashboard-stats` and synthesizes a notification list:

- **Pending appointments** (amber FiClock) → `/dashboard/appointments`
- **New enquiries** (blue FiMessageSquare) → `/dashboard/enquiries`
- **Pending reviews** (blue FiStar) → `/dashboard/reviews`
- **Pending payments** (rose FiCreditCard, admin only) → `/dashboard/payments`
- **Today's appointments** (emerald FiCalendar) → `/dashboard/today`

Closes on outside click. Footer link "Go to Dashboard".

9.9 ExportButtons

Multi-step export wizard. Steps:

1. (If `dateField` provided) **Date range** — From / To inputs that filter the rows.
2. **Column selection** — chips for each `columns[]` definition, with "Select all" / "Clear" controls.
3. **Format** — three large tiles: Excel (`#1f9a4f`), PDF (`#d4434a`), CSV (`#0b67c2`); active tile gets a 2 px blue ring.

Header: blue-600 → indigo-600 gradient with title, subtitle, record count. Calls `exportToPDF` / `exportToExcel` / `exportToCSV` utilities (in `src/utls/`).

Column definition shape:

```
{ key?: 'patient_name', label: 'Patient', map?: (row) => string }
```

9.10 NewAppointmentModal (walk-in booking)

Two modes:

- **auto_slot=true (default)** — server picks the next free slot after the latest booking. Used for true walk-ins. Cash only.
- **auto_slot=false** — admin picks a specific slot from a card-grid of all slots for the date. Can be cash or online.

Form: Full name, mobile (10 digit, auto-strips non-numeric), email, DOB, gender, service dropdown (from `GET /services?active=true`), date picker, slot-mode tabs, slot preview / grid, problem description, payment-mode chips.

Slot grid colors:

- **Available** — `border-dash-line bg-white hover:bg-[#f6f7fb]`
- **Selected** — `border-admin-deep bg-admin-soft text-admin-deep font-semibold`
- **Booked** — `border-rose-200 bg-rose-50 text-rose-500 line-through`
- **Past** — `border-gray-200 bg-gray-100 text-gray-400`

API calls: `POST /appointments/staff-create` , `POST /appointments/verify-payment` (Razorpay JS SDK opens checkout for online mode).

9.11 CancelAppointmentModal

Reason input + refund slider. If paid: defaults `refund=true`, `percent=80` . Quick-pick buttons 50% / 80% / 100%. Live breakdown:

Original amount	₹1,500
Cancellation fee	-₹300
Refund total	₹1,200

If unpaid: shows a notice instead — "Payment isn't captured yet, no refund needed".

API: `POST /appointments/{id}/cancel` with `{ refund, refund_percent?, reason? }` .

9.12 EnquiryDetailsDrawer

Right-side slide-in drawer (`width: min(640px, 96vw)`). Loads `GET /enquiries/{id}` .

Sections:

1. **Header** — avatar with initials, name, ID, status & priority badges, "waited X" pill, "SLA" badge if overdue (>24 h old and status='new').
2. **Contact card** — Email and Mobile rows with action buttons (Email / Copy / Call / WhatsApp).
3. **Subject + Message** — with a Copy button.
4. **Workflow controls** — Status segment buttons (new / contacted / closed); Priority segment buttons (low / normal / high / urgent); Internal notes textarea; Save button. Unsaved indicator: `text-amber-700 bg-amber-50 border-amber-200` .
5. **Quick reply templates** — 4 presets that fill `{first}` (first name) and `{subject}` and open Email / WhatsApp.
6. **Footer** — Delete (left) + Close (right).

Escape closes with unsaved-changes protection. `PATCH /enquiries/{id}` saves `{ status, priority, internal_note }` ; server stamps `responded_at` on first transition from 'new'.

9.13 PatientDetailsDrawer

Right-side slide-in drawer that loads `GET /admin/patients/{id}` . Sections:

1. **Header** — avatar, full name, ID, "Since" date.
2. **Profile card** — Email, Mobile, Gender, DOB rows with action buttons.
3. **Lifetime stats row** — Bookings, Lifetime ₹, Last visit (X days ago), Completed ratio. Each is a StatMini tile.
4. **Tabs** — Appointments / Payments / Notes.
 - **Appointments** — service title, queue # (mono blue pill), date/time, doctor, amount, payment + appointment StatusBadges.
 - **Payments** — amount, Razorpay payment_id, status, paid date.
 - **Notes** — rows from appointments where `internal_note` is non-empty.
5. **Footer** — last updated timestamp + Close.

9.14 AnalyticsCharts (Recharts)

Used only on `DashboardHome` for admin viewers.

Chart	Type	Color	Data shape
RevenueTrendChart	Line + gradient area	Blue #2563eb line, blue-50 → transparent gradient	<code>[{ day, revenue }]</code>
BookingsPerDayChart	Bar	Blue #1f6fd0, 22 px wide, 4 px top radius	<code>[{ day, bookings }]</code>
StatusDonutChart	Donut (inner 50, outer 84)	STATUS_COLORS map	<code>[{ status, count }]</code>
TopServicesChart	Horizontal bar	Teal #0d9488	<code>[{ title, bookings }]</code>

All charts are wrapped in `ChartCard` (`bg-white rounded-xl border border-slate-200` , header with icon-badges + title + subtitle, body height 280 px).

`STATUS_COLORS` :

pending	#d97706	(amber)
confirmed	#2563eb	(blue)
completed	#059669	(emerald)
cancelled	#dc2626	(red)
no_show	#64748b	(slate)
rescheduled	#f59e0b	(amber)

9.15 StatusBadge

Single `<span>` with `badge badge-{value}` classes. Optional `label` prop overrides display text. See §8.1 for color mapping.

9.16 Login

Centered card on `bg-cream`. Spa icon (amber-gradient circle), heading, email/password inputs (`.input-base`), submit button (`.btn-primary` full-width). Posts to Redux `loginThunk`. Auto-redirects to `/dashboard` if already authenticated. Toast error on failure.

10. PAGE-BY-PAGE SPECIFICATION

Standard pattern used by **every** dashboard page:

1. `<PageHeader title=... subtitle=...>{actions}</PageHeader>`
2. (Optional) row of 4 `StatCard`s and/or `MetricTile`s.
3. Filter bar in a `bg-white rounded-xl border border-slate-200 p-4` card.
4. Data table (or grid of cards) in another `rounded-xl border border-slate-200 bg-white` card.
5. Drawer or modal for detail / edit flows.

10.1 Dashboard Home — `/dashboard`

File: `pages/dashboard/DashboardHome.jsx`

Layout:

- Header: "Welcome back, {first name}" + subtitle, with a date chip ("Saturday, 31 May, 2026"), Refresh button, and (admin) Export buttons for today's queue.
- 4 StatCards: **Today's Appointments** (blue), **Pending** (amber), **Confirmed** (emerald), **Completed** (indigo).
- 4 MetricTiles row: **Revenue (Paid)** (admin only, emerald), **Pending Payments** (admin only, amber), **Total / New Enquiries** (blue), **Total / Pending Reviews** (rose).
- (Admin only) 2×2 grid of analytics charts: Revenue Trend, Bookings/Day, Status Donut, Top Services.
- "Today's Queue" card: top 8 rows from `/appointments/today` with View All link.

API:

- GET `/admin/dashboard-stats` or `/staff/dashboard-stats`
- GET `/appointments/today`
- GET `/admin/analytics?days=30` (admin only)

Notes: Skeleton (`DashBoardHomeSkeleton`) shown during initial load. Refresh button reloads all three calls in parallel.

10.2 Today's Bookings — /dashboard/today

File: `pages/dashboard/TodayBookings.jsx`

Layout:

- Header + "New appointment" button (opens `NewAppointmentModal`) + Refresh.
- View tabs: **Active queue** | **Completed today** (with completed count badge).
- Notice (only on Completed view): "Completed bookings are locked — status, refund and cancel actions are disabled to protect revenue."
- Table columns: Queue, Patient (name + mobile + email), Service (title + duration), Time (12-hour), Source (Online / Walk-in + optional "Cash" badge), Amount, Payment status, Appointment status, Actions.

Actions per row (active view):

- **Complete** button (primary, calls `PATCH /appointments/{id}/status` with `appointment_status: 'completed'`, with a confirm prompt — "Once completed it cannot be re-opened, cancelled or refunded").
- Status dropdown (pending / confirmed / no_show / rescheduled).
- Resend confirmation email (`POST /appointments/{id}/resend-confirmation`).
- Cancel (opens `CancelAppointmentModal`).

Actions per row (completed view): "Locked" badge, no actions.

10.3 Appointments — /dashboard/appointments

File: `pages/dashboard/Appointments.jsx`

Layout:

- Header + "New appointment" + Export.
- Range tab bar: All | Today | Tomorrow | This week | This month.
- Filter grid (4 cols on md+): Search (name/mobile/email), Status (pending/confirmed/completed/cancelled/no_show/rescheduled), Payment (pending/paid/failed/refunded), Service.
- Table columns: Queue, Patient, Service, Date, Time, Source, Amount, Payment, Status, Actions.

Actions per row (not completed): status dropdown, Reschedule (opens modal with date + time inputs → `PATCH /appointments/{id}/reschedule`), Cancel.

Completed rows: show "Locked" badge.

API: `GET /appointments?range=&appointment_status=&payment_status=&service_id=&search=` .

10.4 OPD Schedule — /dashboard/opd

File: `pages/dashboard/OpdSchedule.jsx`

Two-column layout (1 col on mobile, 2 cols on lg):

- **Left (editor):**
 - Date picker + "Today" quick button.
 - Source indicator: "Custom hours saved" (green) or "Using default hours" (gray).
 - Clinic state toggles: **Open** / **Closed (holiday)**.
 - Time pickers: Start time / End time (custom `TimePickerAMP` component, 12-hour AM/PM).
 - Slot duration buttons: 10 / 15 / 20 / 30 / 45 / 60 minutes.

- Note input (max 255 chars, "Only staff sees this").
- **Save hours for this date** (primary) and **Use clinic defaults** (visible only when an override exists).
- **Right (preview):**
 - Slot grid (3 cols on mobile, 4 on sm): emerald=available, rose strikethrough=booked, gray=past.
 - Legend + summary.

API:

- GET /opd/day/{date} → returns the resolved schedule + override details.
- PUT /opd/{date} → upsert override.
- DELETE /opd/{date} → remove override (fall back to defaults).
- GET /appointments/slots?date={date} → slot board for the preview.

10.5 Payments — /dashboard/payments (admin only)

File: pages/dashboard/Payments.jsx

Layout:

- Header with totals: # transactions, total paid INR, total refunded INR.
- Filter card (sticky): status tabs (All / Paid / Pending / Failed / Refunded), search (patient name/mobile/order_id/payment_id), method dropdown (card / upi / netbanking / wallet / emi), from-date, to-date.
- Table columns: Patient (name + mobile), Razorpay IDs (order_id + payment_id, monospace), Amount (INR), Refund (emerald, with reason truncated at 28 chars), Method, Status badge, Date.

API: GET /payments?status=&method=&search=&from=&to= .

10.6 Patients — /dashboard/patients (admin only)

File: pages/dashboard/Patients.jsx

Layout:

- Header + Refresh + Export.
- 4 StatCards: Total Patients (blue), New (30 days) (emerald), Female / Male (indigo), Profile Complete % (amber/emerald).
- Filter bar (12-col grid): Search (5), Gender (2), Age band (2 — under 18 / 18-25 / 26-35 / 36-50 / 50+ / no DOB), Joined (2 — 7d / 30d / 90d / 12m), Reset (1).
- Sort row: Newest / Oldest / Name A→Z / Name Z→A / Youngest / Oldest.
- Table columns: Patient (avatar + name + #id button to open drawer), Contact (email + mobile), Age, Gender chip (female=pink-50/pink-700, male=blue-50/blue-700, other=violet-50/violet-700), Joined date, Actions (WhatsApp, Email, View Details).

API:

- GET /admin/patients?search= (server max 50 per page; search server-side, other filters client-side).
- GET /admin/patients/{id} (loaded by PatientDetailsDrawer).

10.7 Enquiries — /dashboard/enquiries

File: pages/dashboard/Enquiries.jsx

Layout:

- Header + Refresh + Export.
- 4 StatCards: Total (blue, with "+N today" trend), New / Awaiting reply (amber, with overdue chip), Avg. response time (emerald), Urgent / High (indigo).
- Filter bar (12-col grid): Search (5), Status (2 — new/contacted/closed), Priority (2 — urgent/high/normal/low), Received (2 — 24h/7d/30d), Reset (1).
- Sort: Newest / Oldest / Priority (urgent first).
- **Bulk action bar** (only when selection exists, `bg-blue-50 border-blue-200`): Mark contacted / Mark closed / Reopen / Delete / Clear.
- Table columns: Checkbox, Enquirer (avatar + name + email + mobile), Subject / message (truncated), Priority chip (`PRIORITY_TONES` : urgent=red, high=amber, normal=slate, low=slate), Status badge + optional "SLA" overdue badge, Waited (humanized: 12m / 2h / 3d), Actions (WhatsApp, Email, View).

API:

- `GET /enquiries?status=&priority=&search=&from=&sort=`
- `GET /enquiries/stats` (total, new_count, contacted_count, closed_count, urgent_count, high_count, overdue_count, last_24h_count, avg_response_minutes)
- `POST /enquiries/bulk/status` / `POST /enquiries/bulk/delete` (admin only)

`EnquiryDetailsDrawer` `PATCHes /enquiries/{id}` with `{ status, priority, internal_note }`.

10.8 Reviews — `/dashboard/reviews`

File: `pages/dashboard/ReviewsManage.jsx`

Layout:

- Status tabs: All | pending | approved | rejected.
- Card grid (1 col mobile, 2 cols desktop) of review cards: patient name, email, 1-5 gold star rating (`fa-solid fa-star`), date, full review text, current status badge + actions (Mark pending / approved / rejected).

API:

- `GET /reviews?status=` (staff/admin) — list all reviews.
- `PATCH /reviews/{id}/status` — update status.

10.9 Services — `/dashboard/services` (admin only)

File: `pages/dashboard/ServicesManage.jsx`

Layout:

- "New service" button toggles an **inline form** above the card grid:
 - Fields: title, slug (lowercase + hyphens, regex `^[a-z0-9-]+$`), short_description (max 280), description (textarea), price (number, ₹), duration_minutes (number), is_active (checkbox), image file.
- Card grid (1/2/3 cols by breakpoint) of service cards: image (or spa icon placeholder), title + price (top-right, `admin-deep`), duration / slug / status (xs text), short description (2-line truncate), Edit + Delete.

API:

- `GET /services` , `POST /services` (multipart), `PATCH /services/{id}` (multipart), `DELETE /services/{id}` .

10.10 Gallery — /dashboard/gallery

File: `pages/dashboard/GalleryManage.jsx`

Layout:

- Upload form with source tabs: **From computer** / **From URL**.
 - From computer: file input (JPG/PNG/WebP/GIF/AVIF/BMP/TIFF, max 20 MB; warning if >1 MB will be auto-compressed to WebP, max 1920 px).
 - From URL: URL input + "mirror" checkbox (download a local copy — recommended for availability).
- Optional title and category.
- Image grid (2/3/4 cols by breakpoint): image (h-44, object-cover), title or "Untitled", category, "linked" badge when no `image_public_id` (i.e., external URL), Active / Hidden toggle button, Delete (admin only).

API:

- `GET /gallery` (public).
- `POST /gallery` (multipart for files; JSON `{ image_url, mirror, title, category }` for URL).
- `PATCH /gallery/{id}` — update `is_active`, title, category.
- `DELETE /gallery/{id}` (admin only) — removes local file if present.

10.11 Staff Manage — /dashboard/staff (admin only)

File: `pages/dashboard/StaffManage.jsx`

Layout:

- "Add staff" button toggles an inline form: full_name (min 2), email, mobile (10 digits, auto-strips non-numeric), password (min 8).
- Table columns: Name, Email, Mobile, Active (Active badge / Disabled badge), Created (date), Actions (Delete with confirm).

API: `GET /admin/staff`, `POST /admin/staff`, `DELETE /admin/staff/{id}`.

10.12 Admin Manage — /dashboard/admins (admin only)

File: `pages/dashboard/AdminManage.jsx`

Identical structure to StaffManage with one addition:

- Current user's row is highlighted with a blue **"You"** badge.
- Delete button is **disabled on your own row** with tooltip.
- Backend also prevents deleting the **last active admin** (returns 409 / error).

API: `GET /admin/admins`, `POST /admin/admins`, `DELETE /admin/admins/{id}`.

10.13 Settings — /dashboard/settings

File: `pages/dashboard/Settings.jsx`

Read-only two-column layout (1 col on mobile):

- **Your profile** card: full_name, email, role, mobile, status (Active / Disabled), user ID.

- **System info** card: Clinic name (Lumière Skin Clinic), Timezone (Asia/Kolkata IST), Frontend stack (React 18 / Vite / Tailwind), Backend stack (Node / Express / MySQL), Payments (Razorpay test mode), Email (Nodemailer + birthday cron @ 09:00 IST).
- Note: "Editable profile + password change ship in the next pass."

No API calls — reads from Redux `selectUser` .

11. BACKEND API REFERENCE

All endpoints are prefixed `/api` . Public endpoints require no auth; staff/admin endpoints require `isAuth` plus `authorizeRoles(...)` .

11.1 Middleware

Middleware	File	Behavior
<code>isAuth</code>	<code>middleware/auth.middleware.js</code>	Reads JWT from cookie or Authorization: Bearer header, verifies, fetches fresh user, attaches to <code>req.user</code> . Returns 401 if user is inactive.
<code>authorizeRoles(...roles)</code>	<code>middleware/role.middleware.js</code>	Checks <code>req.user.role</code> against allowed list. Returns 403 if not allowed.
<code>upload.single('image')</code>	<code>middleware/upload.middleware.js</code>	Multer in-memory; image MIME filter; 20 MB limit; returns 415 on non-image.
<code>notFound / errorHandler</code>	<code>middleware/error.middleware.js</code>	404 JSON for unmatched routes; global error catcher; recognises <code>ER_DUP_ENTRY</code> MySQL code → 409.

11.2 Bootstrap (`backend/index.js`)

- `helmet({ crossOriginResourcePolicy: 'cross-origin' })`
- Static `/uploads` (30-day immutable cache).
- CORS allowlist from `FRONTEND_URL` env var (comma-separated).
- `express.json({ limit: '1mb' })` and `express.urlencoded({ extended: true })` .
- `cookieParser()` .
- Rate limits:
 - **Global API:** 300 req / 15 min.
 - **/api/auth/login:** 20 attempts / 15 min.
- Route mounts: see §11.3 below.
- `GET /api/health` (no auth) — health probe.
- `startBirthdayCron()` — daily 09:00 IST.

11.3 Endpoint Inventory

Auth (`/api/auth`)

Method	Path	Auth	Description
--------	------	------	-------------

POST	/login	—	Login; httpOnly cookie + token in body.
POST	/logout	—	Clear cookie.
GET	/me	isAuth	Fresh user row.
POST	/register	isAuth + admin	Create staff/admin (legacy; admin pages prefer /admin/admins and /admin/staff).

Appointments (/api/appointments)

Method	Path	Auth	Description
GET	/slots?date=YYYY-MM-DD	—	Public slot board (calls <code>opd.controller.getDaySlots</code>).
POST	/create-order	—	Guest booking: validate, upsert patient, create appointment with queue #, create Razorpay order, create payments row (status=pending).
POST	/verify-payment	—	Verify HMAC, confirm with Razorpay fetch, mark paid + confirmed, send confirmation email.
POST	/staff-create	staff/admin	Walk-in/counter booking; auto or manual slot; cash or online.
GET	/stats	staff/admin	today_count, pending_count, confirmed_count, completed_count, paid_count, pending_payment_count, total_revenue.
GET	/today	staff/admin	Today's appointments.
GET	/	staff/admin	All appointments with filters.
GET	/:id	staff/admin	One appointment + joined patient/service/doctor names.
PATCH	/:id/status	staff/admin	Update appointment_status, payment_status, internal_note. Prevents re-opening completed.
PATCH	/:id/reschedule	staff/admin	Change date/time. Validates slot, checks no collision.
POST	/:id/cancel	staff/admin	Refund + cancel. Online → Razorpay refund. Cash → returns cash_refund_due.
POST	/:id/resend-confirmation	staff/admin	Resend confirmation email.
DELETE	/:id	admin	Hard delete (override for completed records).

Payments (/api/payments)

Method	Path	Auth	Description
GET	/	admin	List with status, method, search, from, to.

GET	/:id	admin	Joined with patient.
-----	------	-------	----------------------

Services (/api/services)

Method	Path	Auth	Description
GET	/	—	List (optionally ?active=true).
GET	/:slug	—	One by slug.
POST	/	admin + upload	Create (multipart).
PATCH	/:id	admin + upload	Update (multipart).
DELETE	/:id	admin	Delete + clean up image.

Enquiries (/api/enquiries)

Method	Path	Auth	Description
POST	/	—	Submit contact form.
GET	/stats	staff/admin	Aggregate stats.
GET	/	staff/admin	List with status, priority, search, from, to, sort.
GET	/:id	staff/admin	Detail.
PATCH	/:id	staff/admin	Update status, priority, internal_note; stamp responded_at on first transition.
PATCH	/:id/status	staff/admin	Legacy status-only endpoint.
POST	/bulk/status	admin	Bulk status update.
POST	/bulk/delete	admin	Bulk delete.
DELETE	/:id	admin	Delete.

Reviews (/api/reviews)

Method	Path	Auth	Description
POST	/	—	Submit review (status defaults to pending).
GET	/public	—	Approved reviews only (limit 100).
GET	/	staff/admin	List with optional status.
PATCH	/:id/status	staff/admin	pending/approved/rejected.
DELETE	/:id	admin	Delete.

Gallery (/api/gallery)

Method	Path	Auth	Description
GET	/	—	List with optional ?active=true&category=.
POST	/	staff/admin + upload	Multipart file or JSON { image_url, mirror, title, category }.
PATCH	/:id	staff/admin	Update title, category, is_active.
DELETE	/:id	admin	Delete + clean local file.

Admin (/api/admin) (*all admin-only*)

Method	Path	Description
GET	/dashboard-stats	Combined appointment stats + enquiry/review/patient totals.
GET	/analytics? days=N	7–90 days (default 30): zero-filled bookings & revenue series, status_breakdown, top 6 services.
GET	/patients? search=	List (max 50).
GET	/patients/:id	Profile + lifetime stats + appointment + payment history.
GET	/staff	List staff.
POST	/staff	Create.
PATCH	/staff/:id	Update.
DELETE	/staff/:id	Delete.
GET	/admins	List admins.
POST	/admins	Create.
DELETE	/admins/:id	Delete (with self-protect + last-admin-protect).

Staff (/api/staff)

Method	Path	Auth	Description
GET	/dashboard-stats	staff/admin	Subset of admin stats (no revenue).

OPD (/api/opd)

Method	Path	Auth	Description
GET	/day/:date	—	Resolved schedule + override details.
GET	/defaults	staff/admin	Fetch singleton defaults row (id=1).
PUT	/defaults	staff/admin	Update defaults.
GET	/	staff/admin	List overrides (optionally from=&to=).

PUT	/:date	staff/admin	Upsert override.
DELETE	/:date	staff/admin	Remove override.

12. DATABASE SCHEMA

MySQL 8.0+, InnoDB, utf8mb4. All tables have `created_at` `TIMESTAMP DEFAULT CURRENT_TIMESTAMP` and most have `updated_at` `TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`.

12.1 users

```

id            INT UNSIGNED PRIMARY KEY AUTO_INCREMENT
full_name     VARCHAR(120) NOT NULL
email         VARCHAR(190) UNIQUE
mobile        VARCHAR(20)           -- INDEX
password_hash VARCHAR(255) NOT NULL  -- bcrypt 10 rounds
role          ENUM('admin','staff') DEFAULT 'staff' -- INDEX
gender        ENUM('male','female','other')
dob           DATE
profile_image VARCHAR(500)
is_active     TINYINT(1) DEFAULT 1
created_at    TIMESTAMP
updated_at    TIMESTAMP

```

Indexes: `idx_users_role`, `idx_users_mobile`.

12.2 patients

```

id            INT UNSIGNED PK AUTO_INCREMENT
full_name     VARCHAR(120) NOT NULL
email         VARCHAR(190)
mobile        VARCHAR(20)           -- INDEX
gender        ENUM('male','female','other')
dob           DATE                  -- functional index on (MONTH, DAY) for birthday cron
created_at    TIMESTAMP
updated_at    TIMESTAMP
UNIQUE KEY uq_patients_email_mobile (email, mobile)

```

12.3 services

```

id            INT UNSIGNED PK AUTO_INCREMENT
title         VARCHAR(160) NOT NULL
slug          VARCHAR(190) UNIQUE    -- ^[a-z0-9-]+$
description    TEXT
short_description VARCHAR(280)
price         DECIMAL(10,2) NOT NULL DEFAULT 0

```

```

duration_minutes INT UNSIGNED DEFAULT 30
image_url        VARCHAR(500)           -- Cloudinary URL
image_public_id  VARCHAR(255)           -- Cloudinary public ID
is_active        TINYINT(1) DEFAULT 1   -- INDEX
created_at       TIMESTAMP
updated_at       TIMESTAMP

```

12.4 doctors

```

id INT UNSIGNED PK AUTO_INCREMENT
name VARCHAR(160) NOT NULL
specialization VARCHAR(160)
email VARCHAR(190)
mobile VARCHAR(20)
image_url VARCHAR(500)
image_public_id VARCHAR(255)
is_active TINYINT(1) DEFAULT 1
created_at TIMESTAMP
updated_at TIMESTAMP

```

12.5 appointments

```

id INT UNSIGNED PK AUTO_INCREMENT
patient_id INT UNSIGNED REFERENCES patients(id) ON DELETE RESTRICT
service_id INT UNSIGNED REFERENCES services(id) ON DELETE RESTRICT
doctor_id INT UNSIGNED REFERENCES doctors(id) ON DELETE SET NULL
appointment_date DATE -- INDEX
appointment_time TIME -- composite INDEX with date
queue_number INT UNSIGNED -- atomically assigned per date
problem_description TEXT
appointment_status
ENUM('pending','confirmed','completed','cancelled','no_show','rescheduled')
DEFAULT 'pending' -- INDEX
payment_status ENUM('pending','paid','failed','refunded') DEFAULT 'pending' -- INDEX
payment_mode ENUM('online','cash') DEFAULT 'online'
booking_source ENUM('online','offline') DEFAULT 'online'
amount DECIMAL(10,2) -- snapshot of service price
created_by INT UNSIGNED REFERENCES users(id) ON DELETE SET NULL
internal_note TEXT -- appended on cancellation
created_at TIMESTAMP
updated_at TIMESTAMP
UNIQUE KEY uq_appt_date_queue (appointment_date, queue_number)

```

12.6 opd_defaults (singleton)

```

id TINYINT UNSIGNED PRIMARY KEY -- always = 1
start_time TIME DEFAULT '09:00:00'

```

```

end_time            TIME DEFAULT '18:00:00'
slot_duration_minutes INT UNSIGNED DEFAULT 15      -- BETWEEN 5-120
is_open             TINYINT(1) DEFAULT 1
updated_at          TIMESTAMP

```

12.7 opd_schedules (per-date override)

```

id                  INT UNSIGNED PK AUTO_INCREMENT
opd_date            DATE UNIQUE
start_time          TIME
end_time            TIME                      -- CHECK > start_time
slot_duration_minutes INT UNSIGNED          -- CHECK BETWEEN 5-120
is_open             TINYINT(1)              -- 0 = holiday
note               VARCHAR(255)
created_by          INT UNSIGNED REFERENCES users(id) ON DELETE SET NULL
created_at          TIMESTAMP
updated_at          TIMESTAMP

```

12.8 payments

```

id                  INT UNSIGNED PK AUTO_INCREMENT
appointment_id      INT UNSIGNED REFERENCES appointments(id) ON DELETE CASCADE
patient_id          INT UNSIGNED REFERENCES patients(id) ON DELETE RESTRICT
razorpay_order_id   VARCHAR(120) UNIQUE
razorpay_payment_id VARCHAR(120)           -- INDEX
razorpay_signature  VARCHAR(255)
amount             DECIMAL(10,2)
currency            VARCHAR(8) DEFAULT 'INR'
payment_status      ENUM('pending','paid','failed','refunded') DEFAULT 'pending' -- INDEX
payment_method      VARCHAR(40)           -- netbanking/card/upi/...
paid_at             TIMESTAMP
razorpay_refund_id  VARCHAR(120)
refund_amount       DECIMAL(10,2)
refund_reason       VARCHAR(255)
refunded_at         TIMESTAMP
created_at          TIMESTAMP
updated_at          TIMESTAMP

```

12.9 enquiries

```

id                  INT UNSIGNED PK AUTO_INCREMENT
name               VARCHAR(120) NOT NULL
email              VARCHAR(190) NOT NULL
mobile             VARCHAR(20)
subject            VARCHAR(200)
message            TEXT NOT NULL           -- min 5 chars
status             ENUM('new','contacted','closed') DEFAULT 'new' -- INDEX

```

```

priority      ENUM('low','normal','high','urgent') DEFAULT 'normal' -- INDEX
internal_note TEXT
responded_at  TIMESTAMP                                -- stamped on first non-new
transition
created_at    TIMESTAMP
updated_at    TIMESTAMP

```

12.10 reviews

```

id            INT UNSIGNED PK AUTO_INCREMENT
patient_name  VARCHAR(120) NOT NULL
email         VARCHAR(190)
rating        TINYINT UNSIGNED                    -- CHECK BETWEEN 1-5
review_text   TEXT NOT NULL                        -- min 5 chars
status        ENUM('pending','approved','rejected') DEFAULT 'pending' -- INDEX
created_at    TIMESTAMP
updated_at    TIMESTAMP

```

12.11 gallery

```

id            INT UNSIGNED PK AUTO_INCREMENT
title         VARCHAR(160)
image_url     VARCHAR(500) NOT NULL                -- /uploads/gallery/... OR full external URL
image_public_id VARCHAR(255)                       -- disk path "gallery/abc.webp" (NULL = external
URL only)
category      VARCHAR(80)                          -- INDEX
is_active     TINYINT(1) DEFAULT 1                 -- INDEX
created_at    TIMESTAMP
updated_at    TIMESTAMP

```

12.12 birthday_email_logs

```

id            INT UNSIGNED PK AUTO_INCREMENT
patient_id    INT UNSIGNED REFERENCES patients(id) ON DELETE CASCADE
email         VARCHAR(190)
sent_date     DATE
status        ENUM('sent','failed')
error_message TEXT
created_at    TIMESTAMP
UNIQUE KEY uq_bday_patient_date (patient_id, sent_date)

```

13. BUSINESS WORKFLOWS

13.1 Online (guest) booking flow

1. Visitor opens `/appointment` , picks service + date, sees slot board via `GET /appointments/slots?date=` .
2. Fills name, mobile, email, DOB, problem description, accepts terms.
3. `POST /appointments/create-order` :
 - `assertSlotAllowed(date, time)` validates against OPD schedule (resolved override or default; rejects holidays and past times).
 - Checks no existing booking at that exact slot (uniqueness).
 - Upserts patient by `(email, mobile)` (unique constraint).
 - Within a transaction with row-level locking, computes `queue_number = MAX(queue_number)+1` for that date.
 - Inserts appointment (`status= pending` , `payment_status= pending` , `amount=service.price`).
 - Calls Razorpay `createOrder(amount)` , inserts `payments` row.
 - Returns: appointment id, queue number, amount, razorpay order details.
4. Frontend opens Razorpay checkout (`window.Razorpay`) with the order details.
5. On success, frontend calls `POST /appointments/verify-payment` with `{ razorpay_order_id, razorpay_payment_id, razorpay_signature }` .
6. Server: HMAC-verifies the signature, then `fetchPayment(payment_id)` to confirm status is `captured` or `authorized` . Marks `payments.payment_status='paid'` , `appointments.payment_status='paid'` , `appointments.appointment_status='confirmed'` . Sends confirmation email (best-effort, swallows errors).
7. Frontend redirects to `/appointment/confirmation/:id` showing queue number, date, time, fee.

Race-condition protection: appointments creation runs within a transaction with `FOR UPDATE` row locks; if the unique key on `(appointment_date, queue_number)` collides, the controller retries up to 3 times.

13.2 Walk-in / counter booking flow

1. Admin or staff clicks **New appointment** on `/dashboard/today` or `/dashboard/appointments` .
2. `NewAppointmentModal` opens. Two modes:
 - **auto_slot=true (default)** — server picks the next free slot after the latest active booking; `payment_mode` must be `cash` . Used for "the next person in the queue".
 - **auto_slot=false** — admin picks any specific free slot; `payment_mode` can be `cash` or `online` .
3. `POST /appointments/staff-create` does the same patient upsert + queue assignment as guest booking, but skips Razorpay if cash. For online walk-ins, creates a Razorpay order and the frontend opens checkout, then calls `/verify-payment` .

13.3 Status transitions

Allowed transitions:

```

pending → confirmed | cancelled | no_show | rescheduled | completed
confirmed → completed | cancelled | no_show | rescheduled
rescheduled → confirmed | completed | cancelled
no_show → (terminal – but editable until completed)
completed → (terminal – must hard DELETE if you really need to undo)
cancelled → (terminal)

```

`completed` is a **hard terminal state**. The UI hides all actions on completed rows and shows a "Locked" badge. The server returns 400 if the client tries to PATCH a completed appointment's status.

13.4 Cancellation + refund flow

1. Admin opens `CancelAppointmentModal` for an appointment.
2. Enters reason, optionally adjusts refund percentage (default 80%).
3. `POST /appointments/{id}/cancel` with `{ refund, refund_percent?, reason? }`.
4. Server logic:
 - If appointment is completed → 400 reject.
 - If `refund=false` or payment is not paid → just mark appointment `cancelled`, append reason to `internal_note`. Return.
 - If `payment_mode='online'` and paid → call `razorpay.refundPayment({ payment_id, amount })`. Update `payments` row: `payment_status='refunded'`, `razorpay_refund_id`, `refund_amount`, `refund_reason`, `refunded_at=NOW()`. Update `appointments.payment_status='refunded'`.
 - If `payment_mode='cash'` and paid → no Razorpay call; server returns `cash_refund_due: amount` so admin can return cash manually. `appointments.payment_status` set to `refunded`, `internal_note` updated with "CASH REFUND DUE: ₹X".
 - In all refund paths, `appointments.appointment_status='cancelled'`.
5. Frontend shows toast + updates the row in place (or removes it if filter excludes cancelled).

13.5 OPD Schedule resolution

When a date is queried (e.g., by `/appointments/slots?date=`):

1. Look up `opd_schedules` WHERE `opd_date = :date`. If found → use that row.
2. Otherwise → use `opd_defaults` row `id=1`.
3. If resolved schedule has `is_open=0` → return `is_open: false`, no slots.
4. Otherwise generate slots from `start_time` to `end_time` stepping `slot_duration_minutes`.
5. For each slot, mark `status`:
 - `booked` if an active appointment exists at that time.
 - `past` if `date = today (IST)` and time is earlier than `now (IST)`.
 - `available` otherwise.

13.6 Birthday email cron

`backend/jobs/birthday.job.js`, node-cron schedule `0 9 * * * in Asia/Kolkata`:

1. Find patients with `MONTH(dob)=MONTH(today)` and `DAY(dob)=DAY(today)`.
2. For each, skip if no email or already a row in `birthday_email_logs` for `(patient_id, sent_date=today)`.
3. Send greeting via nodemailer.
4. Insert log row with `status='sent'` or `'failed'`; on failure write `error_message`.

The unique key `(patient_id, sent_date)` prevents double-sends if the cron is rerun.

13.7 Last-admin protection

`DELETE /api/admin/admins/:id`:

- Reject if `req.user.id == :id` (can't delete self).
- Reject if deleting would leave **0 active admins** (`SELECT COUNT(*) FROM users WHERE role='admin' AND is_active=1`).

The frontend's `AdminManage` disables the delete button on the current user's row with a tooltip explanation. The "last admin" rule is enforced server-side only.

13.8 Enquiry SLA (overdue) logic

An enquiry is **overdue** if:

- `status = 'new'`, and
- `created_at < NOW() - 24 hours`.

Server-side `enquiries/stats` returns `overdue_count`. Frontend renders a red "SLA" badge on overdue rows and a quick "Show N overdue" filter button when none of the status/priority filters are active.

14. THIRD-PARTY INTEGRATIONS

14.1 Razorpay

- **Library:** `razorpay` (server SDK), `Razorpay` (browser checkout JS, loaded by script tag on booking pages).
- **Files:** `backend/config/razorpay.js` exports `createOrder`, `verifySignature`, `fetchPayment`, `refundPayment`.
- **HMAC verification** (signature check):

```
const expected = crypto
  .createHmac('sha256', RAZORPAY_KEY_SECRET)
  .update(`${razorpay_order_id}|${razorpay_payment_id}`)
  .digest('hex');
return expected === razorpay_signature;
```

- **Refunds** can be partial (`amount` in paise). Refund ID is logged in `payments.razorpay_refund_id` for audit.

14.2 Nodemailer

- **Files:** `backend/config/mailer.js` exports `sendAppointmentConfirmation(apt)` and `sendBirthdayEmail(patient)`.
- HTML templates likely sit in `config/emailTemplates/`.
- Email failures are swallowed in critical flows (booking, refund) so they never block the user.

14.3 Cloudinary (legacy)

Used by services for image upload. Gallery has moved to local disk storage. May be fully migrated to local disk later — keep both paths supported during transition.

14.4 Sharp (image pipeline)

- `backend/utils/imageStorage.js` exposes:
 - `storeImageBuffer({ folder, buffer, mime })` — compress + content-hash + write.
 - `storeImageFromUrl({ folder, sourceUrl })` — fetch (10 MB ceiling, 15 s timeout, image/* required), then store.

- `removeStoredImage(disk_path)` — best-effort unlink.
- Compression triggers when source is > 1 MB or longest side > 1920 px. Output is WebP @ 80%.
- Filenames are sha1-hashed so re-uploading the same image is idempotent.

15. ENVIRONMENT VARIABLES

Variable	Default	Purpose
PORT	5000	Express server port
NODE_ENV	development	Enables stack traces in error responses
FRONTEND_URL	http://localhost:5173	Comma-separated CORS allowlist
JWT_SECRET	(required)	JWT signing key
JWT_EXPIRES_IN	7d	JWT expiry
COOKIE_SECURE	false	Secure cookie flag (set true in production)
DATABASE_URL	(required)	MySQL DSN
RAZORPAY_KEY_ID	(required)	Razorpay key id (sent to client)
RAZORPAY_KEY_SECRET	(required)	Razorpay secret
DEFAULT_APPOINTMENT_FEE	500	Fallback if service price not set
CLINIC_TIMEZONE	Asia/Kolkata	Used by cron + slot/date math
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS	(required)	Mail server creds
SMTP_FROM_EMAIL, SMTP_FROM_NAME	(required)	Sender address
APP_URL	auto from request	Base URL for image absolutization
CLOUDINARY_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET	optional	If using Cloudinary for service images

Frontend uses `VITE_API_URL` (e.g., `http://localhost:5000`) — axios baseURL becomes `${VITE_API_URL}/api`.

16. SUGGESTED IMPLEMENTATION ORDER

Build the system in this order to minimize blocking dependencies:

1. **DB + Express bootstrap** — tables in §12, `helmet` /CORS/rate limit, `/api/health`.
2. **Auth** — `users` seed (one admin), `POST /auth/login`, `GET /auth/me`, `POST /auth/logout`. JWT in cookie + body. Middleware `isAuth` + `authorizeRoles`.

3. **Frontend bootstrap** — Vite + Tailwind + Redux Toolkit + react-router. Set up `axios.js` , `authSlice` , `ProtectedRoute` , `RoleBasedRoute` , `Login` .
4. **Design system foundation** — `tailwind.config.js` `palette` (§8.1), `index.css` `@layer` components for `.dash-card` , `.dash-input` , `.dbtn-*` , `.badge-*` , `.dash-live-dot` , `.skel` , `.dash-scroll` , `.dash-kbar` .
5. **Shell** — `DashboardLayout` , `Sidebar` , `Topbar` , `navConfig.jsx` . Get the sidebar collapse, command palette stub, and topbar live-dot working.
6. **Reusable components** — `PageHeader` , `Breadcrumbs` , `StatCard` , `MetricTile` , `StatusBadge` , `EmptyState` , `Skeleton` , `ExportButtons` , `CommandPalette` , `NotificationPanel` .
7. **OPD module** — `opd_defaults` , `opd_schedules` , `controller` , `/api/opd` routes, `OpdSchedule.jsx` , `slot generation` (`getDaySlots`). Critical foundation for booking.
8. **Services CRUD** — `services` table, `service.controller.js` , `ServicesManage.jsx` .
9. **Public booking + Razorpay** — `POST /create-order` , `POST /verify-payment` , `appointments` + `payments` tables, queue number assignment.
10. **Appointments pages** — `TodayBookings` , `Appointments` , `NewAppointmentModal` , `CancelAppointmentModal` . Walk-in flow (`/staff-create`).
11. **Dashboard Home** — `/admin/dashboard-stats` , `/admin/analytics` , `charts`.
12. **Patients** — `Patients.jsx` , `PatientDetailsDrawer` , `/admin/patients` , `/admin/patients/:id` .
13. **Enquiries** — table, controller (with `responded_at` logic + SLA stats), `Enquiries.jsx` , `EnquiryDetailsDrawer` , bulk ops.
14. **Reviews** — `reviews` table, controller, `ReviewsManage.jsx` .
15. **Gallery** — `gallery` table, `imageStorage.js` , `GalleryManage.jsx` .
16. **Payments page** — `Payments.jsx` (read-only with filters + export).
17. **Staff/Admin user CRUD** — `StaffManage.jsx` , `AdminManage.jsx` , last-admin protection.
18. **Settings** — `Settings.jsx` (read-only first cut).
19. **Birthday cron** — `birthday.job.js` , `birthday_email_logs` table.
20. **Polish** — toast notifications, skeleton states, export wizards, error boundaries, mobile responsive sweeps.

APPENDIX A — Quick reference: dashboard color cheat sheet

App bg	#f7f8fa	
Card bg	ffffff	
Card border	#e5e7eb	(slate-200)
Body text	#334155	(slate-700)
Muted text	#64748b	(slate-500)
Section label	#94a3b8	(slate-400)
H1 / strong text	#0f172a	(slate-900)
Brand accent	#2563eb	(blue-600)
Brand pressed	#1d4ed8	(blue-700)
Brand soft tint	#eff6ff	(blue-50)
Brand emphasis	#1e40af	(blue-800)
Live / success	#10b981	(emerald-500)
Warning / pending	#d97706 / #b45309	(amber)
Danger / fail	#dc2626 / #b91c1c	(red)
Neutral / closed	#475569 / #f1f5f9	(slate)

APPENDIX B — Quick reference: page anatomy template

```
<div className="space-y-6">
  <PageHeader title="Page" subtitle="...">
    <button className="dbtn dbtn-secondary">Refresh</button>
    <ExportButtons ... />
  </PageHeader>

  { /* Stat cards */ }
  <div className="grid grid-cols-1 sm:grid-cols-2 xl:grid-cols-4 gap-4">
    <StatCard accent="blue" icon ... label="..." value={...} />
    <StatCard accent="amber" icon ... label="..." value={...} />
    <StatCard accent="emerald" icon ... label="..." value={...} />
    <StatCard accent="indigo" icon ... label="..." value={...} />
  </div>

  { /* Filter bar */ }
  <div className="bg-white rounded-xl border border-slate-200 p-4">
    <div className="grid grid-cols-1 md:grid-cols-12 gap-3">
      { /* search col-span=5, two selects col-span=2, range col-span=2, reset col-span=1 */ }
    </div>
    <div className="flex items-center gap-3 mt-3 pt-3 border-t border-slate-100">
      { /* sort + filter chips */ }
    </div>
  </div>

  { /* Data table */ }
  <div className="bg-white rounded-xl border border-slate-200 overflow-hidden">
    <table className="w-full text-sm">
      <thead className="bg-slate-50 border-b border-slate-200">...</thead>
      <tbody className="divide-y divide-slate-100">...</tbody>
    </table>
    <div className="px-5 py-3 border-t border-slate-200 bg-slate-50 ...">{ /* footer */}</div>
  </div>
</div>
```

End of document.